Intro to REACT!

Recap - Client–Server Model & Data Handling

Client–Server Model

1. What is the role of the client in a web application?
2. What is the role of the server?
3. What happens when you type a URL in the browser and press Enter?
4. What is an HTTP request?
5. What is an HTTP response?
6. What does an API actually do?
7. Is an API the same as a server? Why or why not?

Data Handling in Current Projects

1. In your previous projects, where was your data stored?
2. What kind of data structures were you using? (arrays, objects, variables)
3. What happens to that data when the server restarts?
4. What happens to that data if the application crashes?
5. Is that type of storage permanent or temporary?

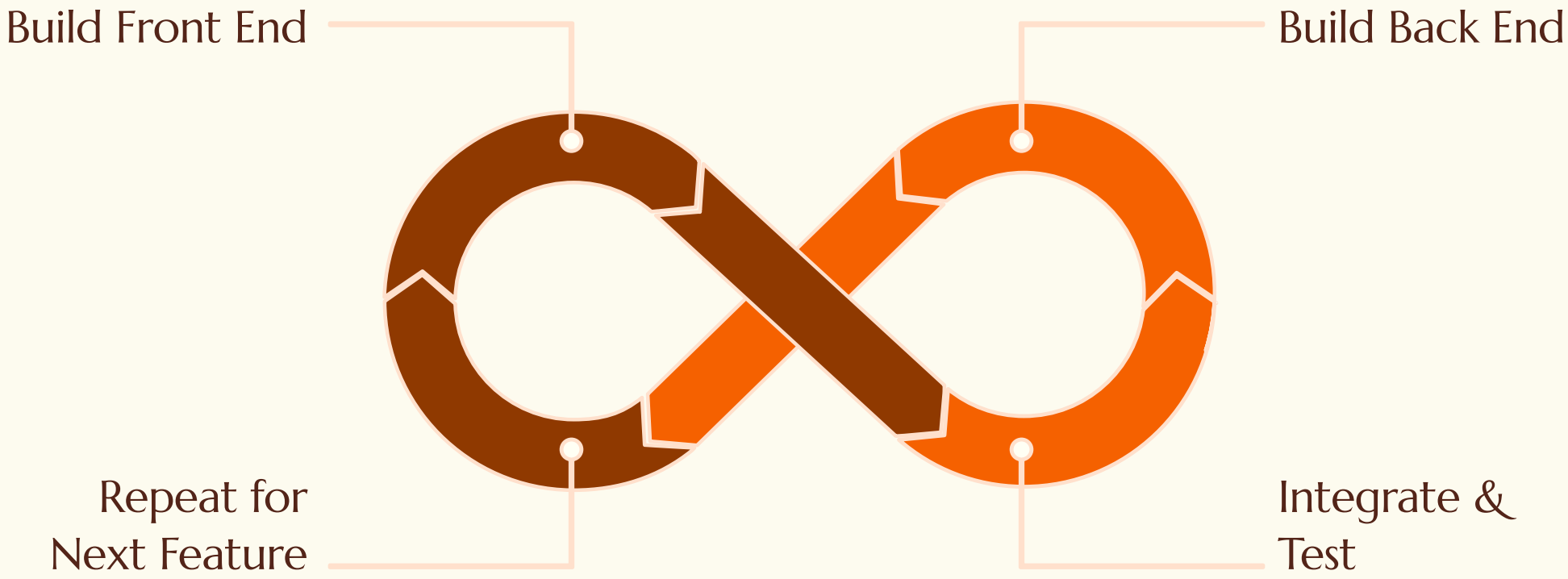
Real-World Thinking

1. Could a social media app function if data disappeared every time the server restarted?
2. Could an e-commerce app work without remembering past orders?
3. Could a banking system rely only on in-memory variables?

Transition to Databases

1. If memory is temporary, what do real applications use?
2. What does "persistent storage" mean?
3. Why do we need something beyond arrays and objects?
4. Where do you think the data physically lives in real-world systems?
5. If we add a third component to Client → Server, what might it be?

Full-Stack: Feature-by-Feature



Start with project setup, then tackle one feature at a time — front end first, then the supporting API. This reduces overwhelm and keeps you thinking from the user's perspective.

Front End First

- What does the user see?
- What do they click?
- What experience are they having?

Then Back End

- Structure the API to support the UI
- Add server logic and data handling

This is **not the only approach.** Teams may design the full backend first, or prototype everything visually. The right method depends on team size, complexity, timeline, and company standards.

What is React?

React lets you create your own custom HTML tags.

React is a JavaScript library for building user interfaces — the part users see and interact with. Its most beginner-friendly superpower: turning reusable logic into custom components.

HTML Tags vs. React Components

Plain HTML — Built-in Only

- , <p>, <button>, <form>
- Cannot invent meaningful tags
- <Navbar> or <UserCard> are not understood

With React — Custom Tags

- Define <Navbar />, <TodoItem />, <UserCard />
- Use them anywhere in your app
- Code stays organized and readable

Why React Over Plain JavaScript?

Components

Define UI once, reuse everywhere. Cleaner, more maintainable code.

Auto UI Updates

React updates the interface automatically when data changes.

Scalability

Structured patterns make large, dynamic, data-driven apps manageable.

UI Libraries

React is the engine. Material UI, Chakra UI, and Ant Design build on top of it.

What is Vite?

The Problem

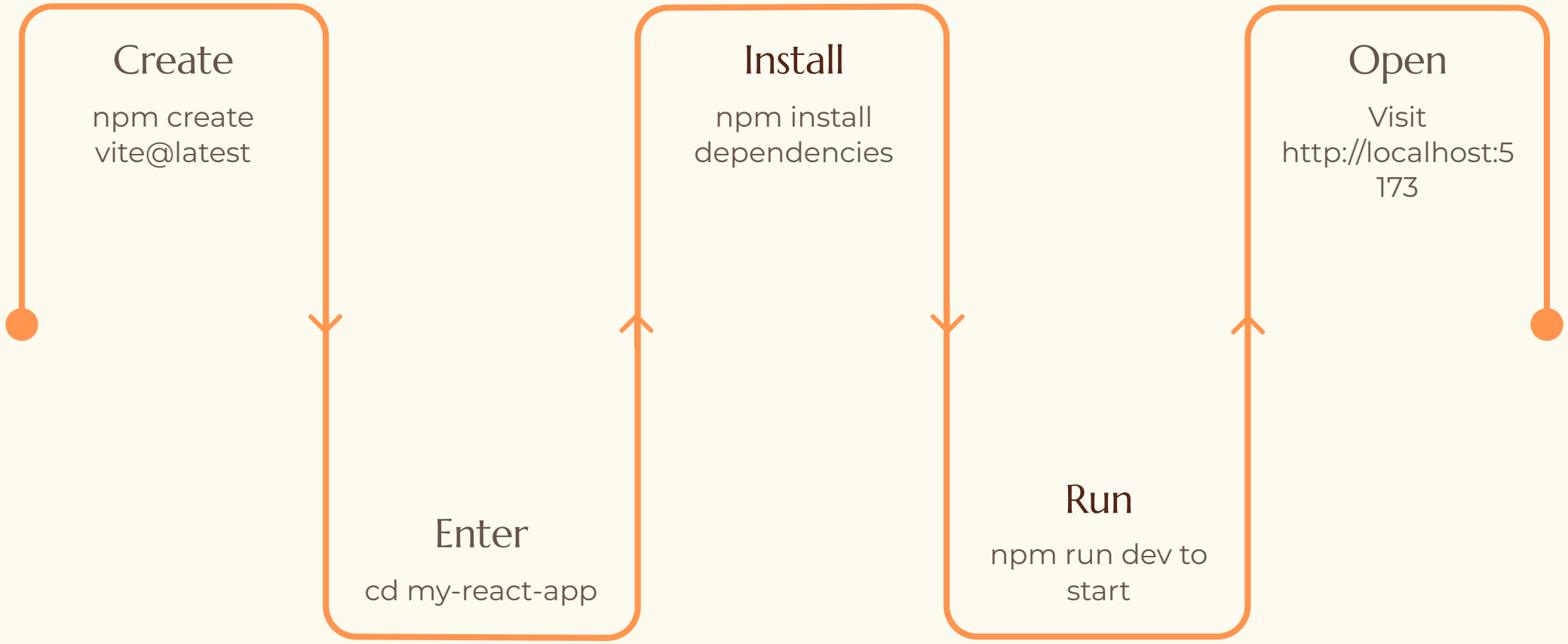
Browsers don't understand JSX or modern JS module syntax directly.

Vite's Role

- Transforms JSX → standard JavaScript
- Handles file imports and modules
- Starts a dev server with hot reload
- Builds an optimized production bundle

React builds the interface.
Vite makes the project run.

Creating Your First React App



After setup, open `src/App.jsx`, remove the default template, and replace it with `return <h1>Hello React</h1>;`. Save and watch the browser refresh automatically.

JSX: HTML Inside JavaScript

What is JSX?

JSX lets you write HTML-like tags inside JavaScript. Everything inside `return ( ... )` is JSX.

Key Rules

- Must return **one parent element**
- Use `{}` to insert JS values
- Use `className` instead of `class`
- Inline style uses an object: `style={{ color: "red" }}`

Quick Mental Model

JSX

HTML-like syntax inside JavaScript

Component

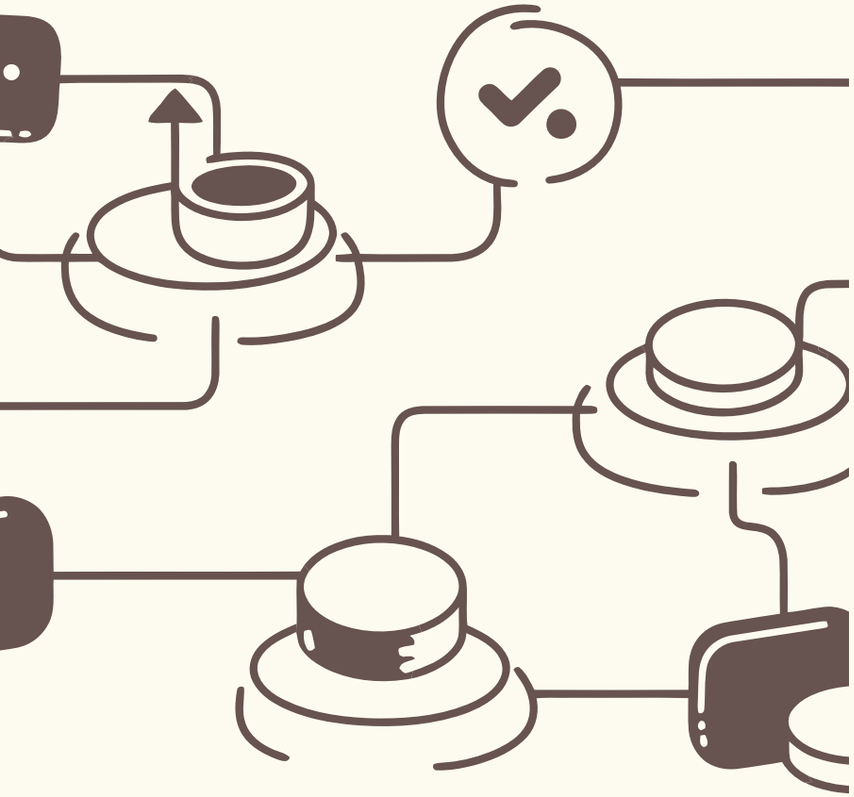
A function that returns JSX

Using a Component

Write it like a custom tag: `<UserProfile />`

Nesting

Place component tags inside other component tags



Components, Props & Nesting

01

Define a Component

A JS function with a **capital letter** name that returns JSX.

02

Pass Props

Send data via attributes: `<UserProfileCard name="Alice" role="Student" />`. Destructure inside for clean access.

03

Import & Use

Import the component file, then use it as a tag anywhere in your app.

04

Nest & Reuse

Render the same component multiple times with different props to build a component tree.